Hybrid-INoT: гибридная агентная архитектура с внутренней ролевой сегрегацией для инженерных задач в условиях длинного общего контекста

Даниил Привезенцев[1*]

[1*]Национальный исследовательский университет «Высшая школа экономики», Москва, Россия.

Corresponding author(s). E-mail(s): daprivezentsev@edu.hse.ru;

Аннотация

Внешняя мультиагентная оркестрация на базе больших языковых моделей многократно передаёт общий контекст независимым ролям и тем самым создаёт существенные издержки координации. В задачах программной инженерии с длинным разделяемым контекстом это приводит к быстрому росту токенного расхода. В работе рассматривается **Hybrid-INoT** — гибридная архитектура, в которой ролевой цикл *planner–worker–critic–self-check* развёрнут внутри одного вызова модели, тогда как вовне вынесены только инструментальные действия, верификация и компрессия контекста. Для этой архитектуры задаются операциональная модель, функция маршрутизации `RouteMode` и условие останова внутреннего ролевого цикла. Показано, что при достаточно длинном общем контексте и наличии нескольких внешних ролей Hybrid-INoT может быть экономичнее Classical-MAS по суммарному токенному расходу; кроме того, получена верхняя оценка расхода токенов в худшем случае и выведено условие экономической эффективности малой модели самопроверки. Помимо этого, в работе предлагаются система метрик токенной эффективности, денежной стоимости и поддерживаемости кода, а также программа эмпирической проверки, включающая пилотное сравнение на HumanEval, абляционный анализ, исследование масштабирования по длине контекста, оценку процедуры самопроверки и расчёт TCO на реальных измерениях.

Keywords: Большие языковые модели, агентные системы, интроспекция ролей, Introspection of Thought, Self-Refine, Reflexion, токенная эффективность, экономика инференса, надёжность кода, экспериментальный протокол

1 Введение

Современный агент для программной инженерии должен последовательно понять постановку задачи, извлечь релевантный контекст из репозитория и построить изменение, которое затем можно проверить тестами или статическим анализом. На практике эти функции часто распределяются между несколькими ролями: одна роль строит план, другая предлагает решение, третья критикует промежуточный результат, четвёртая запускает внешнюю проверку. Такое разбиение удобно концептуально, но становится затратным, когда все роли вынуждены заново читать один и тот же длинный контекст.

В настоящей работе рассматривается именно этот сценарий: задачи, в которых общими для всех ролей оказываются фрагменты кода, документация, сообщения компилятора и журналы выполнения. Нас интересует не только итоговое качество решения, но и цена выбранной схемы оркестрации в токенах, времени ответа и денежной стоимости вызовов. Поэтому далее статья сопоставляет классическую внешнюю мультиагентную организацию с гибридной схемой, в которой ролевая дискуссия частично переносится внутрь одного вызова модели.

1.1 Проблема многократной передачи контекста и оценка её значимости

В агентных системах на основе больших языковых моделей типичной конфигурацией является внешняя мультиагентная оркестрация (Classical-MAS): отдельные процессы или вызовы реализуют роли планировщика, исполнителя, критика и тестировщика [12–14]. В задачах программной инженерии, где несколько ролей читают одни и те же длинные фрагменты кода, логов и документации, такая схема обладает структурным дефектом: *каждая роль получает копию общего контекста* $\mathcal{C}_0$ *заново*, а между ролями дополнительно курсируют сообщения координации. В терминах токенного бюджета это даёт слагаемое

$$D_{\text{ctx}} + H_{\text{coord}} \quad \in \quad \Theta(|\mathcal{A}| \cdot |\mathcal{C}_0|) + \Theta(|\mathcal{A}|^2 \cdot \bar{h}),$$

где $|\mathcal{A}|$ — число ролей, $\bar{h}$ — средняя длина координационного сообщения. Это не умозрительное наблюдение: эмпирические замеры показывают, что при $|\mathcal{A}| \geq 4$ и $|\mathcal{C}_0| \geq 4000$ токенов доля контекста в общем расходе превышает 60% [14, 15]. При стандартных ценах API \$3/M input-токенов (на уровне Claude Sonnet 3.5 / GPT-4o в 2025 г.) цена этого структурного дефекта составляет единицы тысяч долларов в месяц на скромном объёме в 10 000 задач. Здесь D_{ctx} обозначает затраты на повторную передачу общего контекста, H_{coord} — затраты на координационные сообщения, $|\mathcal{C}_0|$ — длину исходного контекста в токенах, $|\mathcal{A}|$ — число ролей, а $\bar{h}$ — среднюю длину координационного сообщения.

Таким образом, издержки рассматриваемой архитектурной схемы допускают количественное выражение: при масштабе 10^4–10^6 инженерных задач Classical-MAS расходует на многократную ретрансмиссию контекста на порядок больше ресурсов, чем на непосредственную генерацию решения. Именно это обстоятельство определяет актуальность рассматриваемой проблемы.

1.2 Существующие подходы и их ограничения

Открытая литература содержит три родственных линии: (i) внутриконтекстная самокритика одной модели — Self-Refine [7], Reflexion [8]; (ii) чередование рассуждения и действия внутри одного агента — ReAct [6], SWE-agent [17]; (iii) внешняя ролевая декомпозиция на отдельные вызовы — MetaGPT [12], ChatDev [13]. Ни одна из этих работ не задаёт *операционально отличимую* промежуточную конфигурацию, в которой (а) внутри одного вызова удерживается разделяемый контекст без дублирования, (б) ролевая сегрегация достаточна для критического разбора, (в) внешними остаются только операции с проверяемыми сигналами, такими как тесты, статический анализ и вызовы инструментов. Отсутствует также формальное условие, при котором такой гибрид доминирует над чисто внешней схемой Classical-MAS по ожидаемому токенному расходу.

1.3 Основные результаты и вклад статьи

В статье получены следующие результаты:

C1. Операциональное определение Hybrid-INoT с явной сигнатурой состояний, функцией маршрутизации `RouteMode` и условием сходимости внутреннего ролевого цикла (§4).

C2. Утверждение 2 о преимуществе Hybrid-INoT над Classical-MAS по ожидаемому токенному расходу при достаточно длинном общем контексте, доказанное через явный подсчёт токенов при выписанных предположениях (§5).

C3. Неравенство (2), задающее условие экономической эффективности малой модели самопроверки (§5).

C4. Система метрик токенной эффективности, стоимости и поддерживаемости кода с обоснованной формой агрегации (§6).

C5. Программа эмпирической валидации, включающая пять исследований E1–E5 с фиксированными базовыми конфигурациями, абляционным анализом и рассмотрением неблагоприятных сценариев (§7).

Эмпирическая проверка преимуществ Hybrid-INoT при длинном контексте обсуждается в §10.

2 Связанные работы и отличие предлагаемого подхода

2.1 От chain-of-thought к внутреннему ролевому дебату

В работе Wei et al. [5] показано, что явная вербализация промежуточных шагов рассуждения может повышать качество решения на задачах, требующих нескольких связанных логических переходов. В данном контексте под промежуточными шагами понимаются текстовые фрагменты, с помощью которых модель раскладывает сложный вывод на последовательность локальных подзадач, а под качеством — итоговая успешность решения целевой задачи. Подход Self-Refine [7]

формализует цикл «генератор–обратная связь–исправление» внутри одной модели; Reflexion [8] дополняет такую схему механизмом рефлексивной памяти, а ReAct [6] связывает внутреннее рассуждение с обращениями к внешним действиям и инструментам. Работа Sun и Zeng [1] описывает *Introspection of Thought* (INoT) как структурированный внутренний дебат нескольких ролей. В то же время Turpin et al. [9] показывают, что промежуточный текст цепочки не следует интерпретировать как достоверное описание скрытого вычислительного процесса. Поэтому в настоящей работе интроспекция рассматривается прежде всего как вычислительный механизм отбора кандидатов, критики и селективного перезапуска, а не как буквальное окно в скрытое состояние модели.

2.2 Мультиагентные системы для программной инженерии

Weiss [10] и Wooldridge [11] задают теоретическую рамку MAS. MetaGPT [12] и ChatDev [13] переносят её в контекст больших языковых моделей: роли реализуются как отдельные вызовы, каждая получает собственную копию контекста. Обзорные работы указывают на два ключевых ограничения Classical-MAS: распространение ошибки по цепочке и рост стоимости координации [14, 15]. SWE-bench и SWE-agent [16, 17] показывают, что практическая автоматизация задач программной инженерии требует длинного контекста и итеративного редактирования, что ещё сильнее обостряет проблему ретрансмиссии D_{ctx}.

2.3 Компрессия контекста и внешняя верификация

LongLLMLingua и LLMLingua-2 [20, 21] демонстрируют, что компрессия промпта сохраняет или улучшает качество на длинном контексте при снижении стоимости. Jia et al. [22] применяют этот подход к коду. Для самопроверки кода недостаточно языковой рефлексии: необходимы внешние наблюдаемые сигналы — тесты, статический анализ и сообщения компилятора [23–25].

2.4 Сопоставление Hybrid-INoT с предшественниками

Для систематизации различий между рассматриваемыми архитектурами выделим пять признаков: организацию контекста, структуру ролевого взаимодействия, способ проверки, механизм выбора режима работы и условие остановки.

Таблица 1 показывает, что отдельные компоненты предлагаемой схемы встречаются в предшествующих работах, однако именно их сочетание в рамках единого внутреннего ролевого цикла, дополняемого внешней инструментальной верификацией и явной маршрутизацией, образует архитектуру Hybrid-INoT.

3 Формальная постановка задачи

3.1 Задача

Пусть $\mathcal{X}$ — множество текстовых спецификаций задач, $\mathcal{C}$ — множество конечных последовательностей контекстных артефактов (файлов кода, фрагментов

Таблица 1 Сравнение Hybrid-INoT с близкими архитектурными классами

Признак	Self-Refine / Reflexion	MetaGPT / ChatDev	Hybrid-INoT
Организация контекста	Один общий контекст без явного ролевого разделения.	Для каждой роли формируется отдельная копия общего контекста.	Один общий контекст сохраняется внутри вызова модели и разделяется между внутренними ролями.
Ролевое взаимодействие	Самокритика встроена в цикл одной модели.	Роли разведены по отдельным вызовам и обмениваются сообщениями.	Планирование, генерация и критика выполняются как внутренние роли в одном цикле.
Проверка решений	Преимущественно языковая самооценка.	Преимущественно ролевая языковая проверка; внешняя верификация зависит от реализации.	Итоговый кандидат проходит обязательную внешнюю инструментальную проверку; языковая критика используется как промежуточный фильтр.
Выбор режима работы	Фиксирован архитектурой метода.	Фиксирован архитектурой метода.	Задаётся явной функцией маршрутизации, переключающей внутренний и внешний режимы.
Условие остановки	Фиксированное число итераций или эпизодическое правило останова.	Завершение конвейера ролей или достижение согласия между ролями.	Остановка после успешной внешней проверки либо после достижения предельного числа итераций.

документации и журналов выполнения), а $\mathcal{Y}$ — множество кандидатных решений, представимых в виде текстовых артефактов и подаваемых на внешнюю проверку. Пусть также $(\Omega, \mathcal{F}, \mathbb{P})$ — вероятностное пространство, индуцированное стохастической генерацией модели, выбором seed и недетерминизмом внешних инструментальных вызовов.

Определение 1 (Агентная инженерная задача)**.** *Агентная инженерная задача — это кортеж $(x, \mathcal{C}_0, \mathcal{Z}, \mathcal{V})$, где $x \in \mathcal{X}$ — текстовое описание задания; $\mathcal{C}_0 \in \mathcal{C}$ — исходный контекст задачи, включающий код, документацию и журналы выполнения; $\mathcal{Z}$ — конечное множество разрешённых внешних инструментов; $\mathcal{V} : \mathcal{Y} \to \{0, 1\}$ — внешняя проверяемая функция успеха, например выполнение тестового набора.*

Стратегия агента индуцирует случайный выход $Y : \Omega \to \mathcal{Y}$. Одновременно определяются случайные величины $T : \Omega \to \mathbb{R}_{\geq 0}$, $C_{\text{inf}} : \Omega \to \mathbb{R}_{\geq 0}$ и $\ell : \Omega \to \mathbb{R}_{\geq 0}$, задающие соответственно суммарный токенный расход, денежную стоимость инференса и задержку выполнения одной задачи. Цель агента состоит в максимизации $\mathbb{E}[\mathcal{V}(Y)]$ при бюджетных ограничениях $\mathbb{E}[T] \leq B$, $\mathbb{E}[C_{\text{inf}}] \leq B_{\$}$, $\mathbb{E}[\ell] \leq B_{\ell}$.

Основные обозначения

$\mathcal{C}_0$ — исходный общий контекст задачи; $|\mathcal{C}_0|$ — его длина в токенах.
$\hat{\mathcal{C}}$ — сжатая версия исходного контекста, используемая во внутреннем цикле.
$\mathcal{A}$ — множество внешних ролей в схеме Classical-MAS; $|\mathcal{A}|$ — число таких ролей.
Y — случайный выход агента со значениями в $\mathcal{Y}$; $y \in \mathcal{Y}$ — отдельный кандидат решения.
K — число кандидатов, порождаемых ролью r_{work} на одной итерации внутреннего цикла.
T — случайная величина суммарного токенного расхода на одну задачу; $\mathbb{E}[T]$ вычисляется по распределению, заданному $(\Omega, \mathcal{F}, \mathbb{P})$.
U_{tok} — показатель токенной эффективности, определяемый как отношение итогового качества решения к суммарному токенному расходу.
$\tau^\star$ — пороговая длина исходного контекста, начиная с которой Hybrid-INoT получает преимущество по ожидаемому токенному расходу.
δ_{H1} — минимальный практически значимый прирост показателя U_{tok}, используемый в гипотезе H1.
C^S_{check} — стоимость предварительной проверки, выполняемой малой моделью.
C^L_{rerun} — стоимость повторного запуска крупной модели.
ρ_0, ρ_1 — вероятности дорогого повторного запуска без предварительной проверки и с предварительной проверкой соответственно.

3.2 Гипотезы исследования

H1 *(преимущество при длинном контексте).* Существует пороговая длина исходного контекста $\tau^\star$ такая, что при $|\mathcal{C}_0| > \tau^\star$ архитектура Hybrid-INoT обеспечивает рост показателя U_{tok} не менее чем на $\delta_{H1} = 15\%$ по сравнению с Classical-MAS, при этом качество решения не ухудшается более чем на 2 п.п.

Проверка: эксперимент E1 (§7.2) с парным критерием Вилкоксона, 95% bootstrap-доверительными интервалами и поправкой Холма–Бонферрони; значение $\tau^\star$ оценивается в E3 (§7.4).

H2 *(экономическая оправданность малой модели самопроверки).* Существует режим, в котором предварительная проверка, выполняемая малой моделью, уменьшает вероятность дорогостоящего повторного запуска настолько, что выполняется условие (2): $C^S_{\text{check}} < (\rho_0 - \rho_1) C^L_{\text{rerun}}$.

Проверка: E4 (§7.5) с прямыми измерениями $\rho_0, \rho_1, C^S_{\text{check}}, C^L_{\text{rerun}}$ на 100 задачах HumanEval и 50 задачах SWE-bench Lite.

H3 *(граница применимости).* При задачах с истинной независимостью подзадач и высокой степенью параллелизма инструментов преимущество Hybrid-INoT по времени ответа исчезает или становится отрицательным.

Проверка: специальный эксперимент E2c (§7.3) на подмножестве задач с явно параллельной структурой из Synthetic Tool-Use Suite.

При проверке H1 во всех сравниваемых конфигурациях фиксируются базовая модель, набор разрешённых инструментов, протокол внешней проверки, состав задач и лимит числа итераций; меняется только схема оркестрации и способ

повторного использования общего контекста между ролями. Под истинной независимостью подзадач в H3 понимаются случаи, когда частичные решения можно строить без обмена промежуточным состоянием и без совместного чтения общего контекста, а высокая степень параллелизма инструментов означает, что внешние проверки и обращения к инструментам могут выполняться одновременно без блокирующих зависимостей. Гипотеза H3 задаёт границу применимости архитектуры: при таких условиях выигрыш внутреннего ролевого цикла по задержке может исчезать. Порог $\delta_{H1} = 15\%$ для H1 выбран как минимально практически значимое улучшение с учётом типичной вариативности токенного бюджета при изменении seed ($\approx 5{-}8\%$ std в [7]) и инженерных издержек развёртывания гибридной схемы при масштабе 10^4 задач в месяц. Устойчивость выводов к выбору этого порога оценивается в E1 на сетке $\delta \in \{5\%, 10\%, 15\%, 20\%\}$.

4 Метод: Hybrid-INoT

4.1 Состояния и сигнатуры ролей

Определим состояние системы $\sigma = (x, \hat{\mathcal{C}}, \mathcal{H}, i, b)$, где $\hat{\mathcal{C}}$ — сжатый контекст, $\mathcal{H}$ — история итераций, $i \in \{0, \dots, I\}$ — счётчик итераций, $b \in \mathbb{N}$ — остаток токенного бюджета.

Пусть Π — пространство планов, а $K \in \mathbb{N}$ — число кандидатов, порождаемых ролью r_{work} на одной итерации. Тогда каждая роль задаётся сигнатурой $r : \mathcal{S}_r^{\text{in}} \to \mathcal{S}_r^{\text{out}}$:

$$
\begin{aligned}
r_{\text{plan}} &: (x, \hat{\mathcal{C}}, b) \mapsto \pi \in \Pi, \\
r_{\text{work}} &: (\pi, \hat{\mathcal{C}}, K) \mapsto \mathbf{y} = (y_1, \dots, y_K) \in \mathcal{Y}^K, \\
r_{\text{crit}} &: (\mathbf{y}, \hat{\mathcal{C}}) \mapsto (s_1, \dots, s_K) \in [0,1]^K, \\
r_{\text{self}} &: (y \in \mathcal{Y}) \mapsto (v, M) \in \{0,1\} \times [0,1].
\end{aligned}
$$

Роли $r_{\text{plan}}, r_{\text{work}}, r_{\text{crit}}$ реализуются как ролевые префиксы внутри *одного* вызова крупной модели над общим $\hat{\mathcal{C}}$; роль r_{self} — внешний процесс запуска тестов и статического анализа.

4.2 Функция маршрутизации

Определение 2 (RouteMode)**.** *Пусть $\varphi_{\text{tool}}(x) \in \{0,1\}$ указывает на обязательность внешнего инструмента, $\varphi_{\text{par}}(x) \in \{0,1\}$ — на доминирующий параллелизм подзадач, $|\hat{\mathcal{C}}|$ — длина сжатого контекста. Тогда*

$$
\texttt{RouteMode}(x, \hat{\mathcal{C}}) = \begin{cases} \texttt{external}, & \varphi_{\text{tool}}(x) = 1 \ \vee \ \varphi_{\text{par}}(x) = 1 \ \vee \ |\hat{\mathcal{C}}| < \tau^{\star}_{\text{route}}, \\ \texttt{internal}, & \text{иначе.} \end{cases}
$$

Параметры $\varphi_{\text{tool}}, \varphi_{\text{par}}$ определяются отдельным классификатором над x (в базовой реализации — лёгким классификатором на основе инструкции к модели,

в дальнейшем — обученным бинарным классификатором); порог $\tau^{\star}_{\text{route}} = 1024$ *токена используется как исходная конфигурация и уточняется в E3.*

4.3 Внутренний цикл и условие сходимости

Algorithm 1 Hybrid-INoT: внутренний интроспективный цикл

Require: задача x, контекст $\mathcal{C}_0$, бюджет B, максимум итераций I, число кандидатов K

Ensure: решение $y^{\star}$, флаг $v^{\star}$, оценка $M^{\star}$

$\mathcal{C} \leftarrow \texttt{PolicySanitize}(x, \mathcal{C}_0)$
$\hat{\mathcal{C}} \leftarrow \texttt{CompressIfNeeded}(\mathcal{C})$ ▷ Алг. 2
$m \leftarrow \texttt{RouteMode}(x, \hat{\mathcal{C}})$ ▷ Опр. 2
if $m = \texttt{external}$ **then return** $\texttt{ExternalAgentWorkflow}(x, \hat{\mathcal{C}})$
end if
$s^{\text{best}} \leftarrow -\infty;\ y^{\text{best}} \leftarrow \perp$
for $i = 1$ to I **do**
  $\pi_i \leftarrow r_{\text{plan}}(x, \hat{\mathcal{C}}, B - \text{spent})$
  $\{y_{i,k}\} \leftarrow r_{\text{work}}(\pi_i, \hat{\mathcal{C}}, K)$
  $(s_{i,1}, \ldots, s_{i,K}) \leftarrow r_{\text{crit}}(\{y_{i,k}\}, \hat{\mathcal{C}})$
  $k^{\star} \leftarrow \arg\max_k s_{i,k};\ y_i \leftarrow y_{i,k^{\star}}$
  if $s_{i,k^{\star}} > s^{\text{best}}$ **then** $s^{\text{best}} \leftarrow s_{i,k^{\star}};\ y^{\text{best}} \leftarrow y_i$
  end if
  $(v_i, M_i) \leftarrow r_{\text{self}}(y_i)$ ▷ внешние тесты и статический анализ
  if $v_i = 1$ **then return** $(y_i, 1, M_i)$
  end if
  $\hat{\mathcal{C}} \leftarrow \texttt{UpdateContextWithFailures}(\hat{\mathcal{C}}, v_i)$
  if $\text{stagnation}(s_{i,k^{\star}} - s_{i-1,k^{\star}}) < \varepsilon$ **then break** ▷ см. Утв. 1
  end if
end for
return $(y^{\text{best}}, 0, M(y^{\text{best}}))$

Утверждение 1 (Терминация внутреннего цикла)**.** *Пусть каждая операция внутри одной итерации алгоритма 1 завершается за конечное время, а число итераций ограничено сверху параметром* I*. Тогда алгоритм завершится не позднее чем после* I *итераций: он либо вернёт решение сразу после успешной внешней проверки* $v_i = 1$*, либо остановится раньше по правилу* $\text{stagnation}(s_{i,k^{\star}} - s_{i-1,k^{\star}}) < \varepsilon$*, либо после исчерпания лимита итераций вернёт лучший найденный кандидат.*

Доказательство Внешний цикл по индексу i содержит не более I проходов. Каждая итерация включает конечный набор шагов: построение плана, генерацию K кандидатов, критическую оценку, внешнюю проверку и обновление контекста. По предположению

все эти операции завершаются за конечное время, следовательно, завершается и любая отдельная итерация. Поэтому алгоритм либо прекращается досрочно по инструкции **return** или **break**, либо после I-й итерации достигает последней инструкции **return**. Тем самым время работы внутреннего цикла сверху ограничено конечным числом итераций, а утверждение доказано. □

Замечание

Утверждение 1 гарантирует только завершение алгоритма, но не монотонный рост оценок s_i и не корректность самого критика. Сценарий, в котором критик систематически подтверждает неверные решения, рассматривается отдельно в E2b (§7.3) как одна из угроз валидности.

4.4 Компрессия контекста

Algorithm 2 Компрессия длинного контекста

Require: $\mathcal{C}_0$, порог длины τ_{len}
Ensure: $\hat{\mathcal{C}}$

1: **if** $|\mathcal{C}_0| \leq \tau_{\text{len}}$ **then return** $\mathcal{C}_0$
2: **end if**
3: разбить $\mathcal{C}_0$ на смысловые блоки $\{b_j\}$
4: оценить $\text{rel}(b_j)$ с помощью классификатора по типу LLMLingua-2 [21]
5: удалить дубликаты, низкоинформативные логи, вторичную документацию
6: сохранить API-сигнатуры, трассы с ошибками, изменяемые файлы и ограничения задачи
7: **return** $\hat{\mathcal{C}}$ с $|\hat{\mathcal{C}}| \leq \tau_{\text{len}}$

Algorithm 3 Селективный перезапуск по подтверждённым сигналам

Require: кандидаты $\{y_1, \ldots, y_K\}$, оценки $\{s_k\}$, бюджет повторов R
Ensure: $y^\star$

1: отсортировать $\{y_k\}$ по убыванию s_k; оставить верхние $M \leq K$
2: **for** каждый y_m **do**
3: $\quad q_m \leftarrow \texttt{ExternalVerify}(y_m)$
4: $\quad$ **if** $q_m = 1$ **then return** y_m
5: $\quad$ **end if**
6: **end for**
7: **if** $R > 0$ **then** промпт формируется только по подтверждённым сигналам ошибки; **return** $\texttt{SelectiveRerun}(R-1)$
8: **end if**
9: **return** $\arg\max_k s_k$

4.5 Селективный перезапуск

5 Аналитическая модель токенных и стоимостных затрат

5.1 Структура аналитических соотношений

Анализ в данном разделе включает два типа соотношений. К первому относятся прямые балансовые равенства и неравенства, задающие структуру токенных и денежных затрат. Ко второму относятся утверждения о сравнении архитектур при фиксированных предположениях относительно длины контекста, стоимости координации и внутренней критики. Такое разделение позволяет отличить общие расчётные соотношения от результатов, требующих дополнительных условий.

5.2 Токенные бюджеты Classical-MAS и Hybrid-INoT

Предположение 1. *Среди ролей $\mathcal{A}$ в схеме Classical-MAS не менее $\alpha|\mathcal{A}|$ ролей, $\alpha \in (0,1]$, читают общий контекст $\mathcal{C}_0$ целиком.*

Предположение 2. *Длина координационных сообщений ограничена $\bar{h}$, число итераций обеих схем равно I, стоимость компрессии $T_{cmp} \le \gamma|\mathcal{C}_0|$ с $\gamma < \alpha|\mathcal{A}|$, стоимость внутренней критики $T^{add}_{core} \le \beta|\hat{\mathcal{C}}|$, $\beta < \alpha|\mathcal{A}| - \gamma$.*

Утверждение 2 (Токенное преимущество при длинном контексте)**.** *При Предположениях 1–2 и $|\mathcal{A}| \ge 3$ существует $\tau^\star > 0$ такой, что при $|\mathcal{C}_0| > \tau^\star$ выполняется*

$$\mathbb{E}[T_{MAS}] - \mathbb{E}[T_{INoT}] \;\ge\; (\alpha|\mathcal{A}| - \gamma - \beta)\cdot|\mathcal{C}_0| + \Theta(|\mathcal{A}|^2\bar{h}) - T_{check} - \mathbb{E}[T_{rerun}] \;>\; 0.$$

Вывод из предположений Обозначим через T_{gen} совокупный токенный вклад генерации кандидатов и иных членов, не зависящих линейно от числа внешних ролей. Тогда первое неравенство получается прямым разложением токенного бюджета Classical-MAS на три части: повторную передачу общего контекста, координационный обмен между ролями и генерацию кандидатов. По Предположению 1 в каждой из I итераций не менее $\alpha|\mathcal{A}|$ внешних ролей перечитывают полный контекст $\mathcal{C}_0$, поэтому

$$T^{\text{MAS}}_{\text{ctx}} \ge I \cdot \alpha|\mathcal{A}| \cdot |\mathcal{C}_0|.$$

По Предположению 2 координационные сообщения дают добавочный член порядка $I \cdot |\mathcal{A}|^2\bar{h}$. Следовательно,

$$\mathbb{E}[T_{\text{MAS}}] \ge I \cdot \alpha|\mathcal{A}| \cdot |\mathcal{C}_0| + I \cdot |\mathcal{A}|^2\bar{h} + T_{\text{gen}}.$$

Для Hybrid-INoT общий контекст сжимается один раз, так что $T_{\text{cmp}} \le \gamma|\mathcal{C}_0|$; дополнительная стоимость внутренней критики ограничена сверху величиной $T^{\text{add}}_{\text{core}} \le \beta|\hat{\mathcal{C}}| \le \beta|\mathcal{C}_0|$. При этом внешняя проверка и возможные повторные запуски образуют слагаемые T_{check} и $\mathbb{E}[T_{\text{rerun}}]$, а прочий генерационный вклад вновь поглощается в T_{gen}. Поэтому

$$\mathbb{E}[T_{\text{INoT}}] \le I \cdot |\hat{\mathcal{C}}| + T_{\text{cmp}} + T^{\text{add}}_{\text{core}} + T_{\text{check}} + \mathbb{E}[T_{\text{rerun}}] + T_{\text{gen}}, \qquad |\hat{\mathcal{C}}| \le |\mathcal{C}_0|.$$

Подставляя указанные оценки и группируя члены, линейно зависящие от $|\mathcal{C}_0|$, получаем нижнюю оценку на разность токенных бюджетов. При достаточно большом $|\mathcal{C}_0|$ линейный член с положительным коэффициентом компенсирует постоянные добавки, что и даёт существование порога $\tau^\star$. □

Область применимости утверждения

Утверждение 2 задаёт нижнюю оценку разности ожидаемых токенных бюджетов при Предположениях 1–2. При $|\mathcal{A}| = 2$ (например, plan+exec без выделенного критика) знак этой разности может изменяться. Аналогично, при $|\mathcal{C}_0| < \tau^\star$ преимущество Hybrid-INoT не гарантируется. Значение $\tau^\star$ зависит от $\alpha, \beta, \gamma, |\mathcal{A}|, I$ и должно уточняться эмпирически.

5.3 Верхняя граница токенного расхода

Утверждение 3 (Верхняя граница расхода токенов Hybrid-INoT)**.** *В худшем случае, когда все I итераций заканчиваются $v_i = 0$,*

$$T_{INoT}^{worst} \leq T_{cmp} + I \cdot K \cdot (p^L + r^L) + I \cdot T_{check} + I \cdot T_{rerun}^{unit}.$$

Полученная оценка задаёт верхнюю границу дополнительных токенных затрат и используется как ограничение при бюджете B: при $T \to B$ алгоритм 1 возвращает лучшего из доступных кандидатов.

5.4 Условие экономической эффективности малой модели самопроверки

Денежная стоимость инференса:

$$C_{\text{inf}} = c_L^{\text{in}} \sum_i p_i^L + c_L^{\text{out}} \sum_i r_i^L + c_S^{\text{in}} \sum_i p_i^S + c_S^{\text{out}} \sum_i r_i^S. \tag{1}$$

Малая модель самопроверки экономически эффективна тогда и только тогда, когда

$$C_{\text{check}}^S < (\rho_0 - \rho_1) \cdot C_{\text{rerun}}^L. \tag{2}$$

Левая часть неравенства задаёт стоимость предварительной проверки, а правая — ожидаемую экономию за счёт снижения вероятности дорогостоящего повторного запуска крупной модели. Неравенство (2) задаёт критерий выбора между дешёвой предварительной проверкой и повторным запуском крупной модели. Его практическая применимость определяется величинами $\rho_0, \rho_1, C_{\text{check}}^S$ и C_{rerun}^L, которые подлежат оценке в E4.

6 Система метрик

6.1 Основные метрики и обоснование формы агрегации

$$U_{\text{tok}} = \frac{Q}{T}, \qquad Q_{\$} = \frac{Q}{C_{\text{inf}}}, \qquad \text{VCR} = \frac{1}{N}\sum_i \mathbb{I}[\text{tests}_i = 1 \wedge \text{static}_i = 1 \wedge \text{sec}_i = 1]. \tag{3}$$

Для неоднородного качества используется взвешенная сумма по классам задач $Q = \sum_b w_b Q_b$ с весами, заданными в таблице 2.

6.2 Агрегированная метрика успешности и её форма

Для агрегированной оценки успешности используется форма

$$\text{MAS}_i = \min(\text{Success}_i, \text{Verified}_i) \cdot M_i^{\lambda}, \qquad \lambda \in [0,1], \quad \lambda = 0.5. \tag{4}$$

Обоснование этой конструкции состоит в следующем. Во-первых, величины Success и Verified являются необходимыми условиями засчитывания решения: patch, не прошедший внешнюю проверку, не должен получать высокий итоговый балл. Во-вторых, множитель M_i^{λ} моделирует влияние поддерживаемости на итоговую оценку; параметр λ регулирует силу этого влияния. В базовой конфигурации используется $\lambda = 0.5$, а устойчивость выводов к выбору λ оценивается в E2d.

6.3 Веса поддерживаемости и их обоснование

$$M_i = w_{\text{cc}}(1 - \widehat{\text{CC}}_i) + w_{\text{dup}}(1 - \widehat{\text{Dup}}_i) + w_{\text{warn}}(1 - \widehat{\text{Warn}}_i) + w_{\text{doc}}\widehat{\text{Doc}}_i. \tag{5}$$

Таблица 2 Веса поддерживаемости и их связь с подкатегориями ISO/IEC 25010:2023 [26]

Параметр	Значение	Обоснование
w_{cc}	0.35	ISO 25010 Modifiability; цикломатическая сложность [27] — наиболее прямой прокси стоимости сопровождения.
w_{dup}	0.25	ISO 25010 Reusability; дублирование — ключевой сигнал технического долга.
w_{warn}	0.20	ISO 25010 Analysability; warnings — поверхностный, но надёжный сигнал.
w_{doc}	0.20	ISO 25010 Modifiability; покрытие документацией как прокси понятности.

Анализ чувствительности

В E2d проверяется устойчивость ранжирования конфигураций B0–B3 при вариации $(w_{\text{cc}}, w_{\text{dup}}, w_{\text{warn}}, w_{\text{doc}})$ в $\pm 20\%$ и при $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$ в (4). Инверсия ранжирования при таких вариациях рассматривается как признак чувствительности метрики к выбранной параметризации.

6.4 Метрики надёжности

$$\mathrm{SCR} = \frac{\#\{\text{ошибочные кандидаты, выявленные процедурой самопроверки}\}}{\#\{\text{все ошибочные кандидаты}\}}, \quad (6)$$

$$\mathrm{RFR} = \frac{\#\{\text{патчи без отката}\}}{\#\{\text{применённые патчи}\}}. \quad (7)$$

7 Программа эмпирической валидации

7.1 Общие правила

Во всех экспериментах фиксируются идентификаторы задач, значения seed, версии моделей с идентификаторами снимков и цены токенов с датой измерения. Для закрытых API без фиксированного идентификатора версии данные выносятся в отдельное приложение с описанием наблюдаемых характеристик. Статистическая обработка включает парный критерий Вилкоксона для непрерывных метрик, критерий Мак-Немара для бинарных исходов, 95% bootstrap-доверительные интервалы (10^4 ресэмплирований) и поправку Холма–Бонферрони.

7.2 E1 — Пилот на HumanEval

Цель: проверить H1 на ограниченной выборке. **Данные:** HumanEval (164 задачи) [18]; для каждой задачи рассматриваются четыре версии с искусственным историческим контекстом длиной $\{512, 2048, 8192\}$ токенов, сгенерированным детерминированным шаблоном; всего $164 \times 4 = 656$ случаев. Значения seed: $\{42, 123, 456, 789, 1337\}$. **Сравниваемые конфигурации:**

- B0 Single-Large (один вызов крупной модели без итераций);
- B1 Single-Large + Self-Refine [7], три итерации;
- B2 Classical-MAS (planner, worker, critic — отдельные вызовы, архитектура в духе MetaGPT);
- B3 Hybrid-INoT ($K = 3$, $I = 3$).

Метрики: pass@1 (доля задач, решённых с первой попытки), T (суммарный токенный расход), U_{tok}, C_{inf} (долл. США на 1000 задач при ценах от {дата}), $Q_{\$}$, время ответа. **Критерий успеха H1:** B3 даёт $\Delta U_{\mathrm{tok}} \geq 15\%$ по сравнению с B2 на подмножестве с $|\mathcal{C}_0| > 2048$, $p < 0.05$ после Holm–Bonferroni; значение pass@1 не ухудшается более чем на 2 п.п. **Критерий неудачи:** при невыполнении этих условий H1 считается неподтверждённой, а выводы формулируются без утверждения о преимуществе Hybrid-INoT.

Таблица 3 задаёт формат представления результатов после выполнения эксперимента.

7.3 E2 — Абляционный анализ и неблагоприятные сценарии

E2a. B3 vs B3a (без компрессии), B3b (без критика), B3c (без самопроверки). Данные: HumanEval, $n = 164$. Критерий: каждый компонент должен давать

Таблица 3 Формат представления результатов эксперимента E1

Конфигурация	pass@1	токены/задача	U_{tok}	долл. США/1000
B0	—	—	—	—
B1 Self-Refine	—	—	—	—
B2 Classical-MAS	—	—	—	—
B3 Hybrid-INoT	—	—	—	—

статистически значимый вклад; если $\Delta_{\text{B3}-\text{B3b}}$ незначимо, роль критика исключается из архитектуры. **E2b** (неблагоприятный сценарий — систематическая ошибка критика). Подмножество из 50 задач, где базовая конфигурация B0 ошибается; измеряется, как часто r_{crit} подтверждает неверного кандидата; порог тревоги: $> 20\%$ означает необходимость внешнего обучения критика. **E2c** (неблагоприятный сценарий — параллельные инструменты, H3). 50 задач с явно независимыми подзадачами из Synthetic Tool-Use Suite; сравнивается время ответа B3 и B2; ухудшение времени ответа B3 до 30% считается допустимым. **E2d** (анализ чувствительности метрик). Ранжирование конфигураций при $w \pm 20\%$, $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$.

7.4 E3 — Масштабирование по длине контекста

Цель: эмпирически оценить $\tau^\star$ из Утв. 2. **Данные:** 50 задач HumanEval $\times$ 4 длины контекста $\{256, 1024, 4096, 16384\} = 200$ случаев. **Метод:** построить $U_{\text{tok}}(|\mathcal{C}_0|)$ для B0, B2, B3; найти точку пересечения $U^{B3}_{\text{tok}}(|\mathcal{C}_0|) = U^{B2}_{\text{tok}}(|\mathcal{C}_0|)$. **Критерий:** $\tau^\star$ должен быть конечен и < 16384; иначе H1 и Утв. 2 не подтверждаются на практически значимом диапазоне.

7.5 E4 — Оценка процедуры самопроверки и H2

Цель: проверить (2). **Данные:** 100 задач HumanEval + 50 SWE-bench Lite [16]. **Измерения:** ρ_0 (доля неверных решений без самопроверки), ρ_1 (с самопроверкой), C^S_{check}, C^L_{rerun}, полученные прямыми измерениями. **Дополнительно:** полнота SCR и соотношение стоимости и полезности процедуры самопроверки. **Критерий H2:** $C^S_{\text{check}} < (\rho_0 - \rho_1) C^L_{\text{rerun}}$ на обоих датасетах с 95% CI.

7.6 E5 — TCO на реальных данных

Цель: заменить синтетический расчёт TCO (старая Таблица 4) наблюдаемыми измерениями. **Данные:** $N = 500$ задач разной длины контекста, B2 и B3. **Измерения:** входные и выходные токены каждой роли, число вызовов API, число перезапусков. C_{inf} определяется по актуальным ценам, для которых фиксируются версия модели и дата измерения. **Экстраполяция:** к 10 000 задач/месяц с CI, *не* с точечной оценкой. **Анализ чувствительности:** изменение цены токена на $\pm 50\%$. **Критерий:** если даже нижняя граница CI $\Delta\text{TCO} < 0$, заявить выгоду нельзя.

8 Экономическая модель

8.1 Формализация TCO

$$\mathrm{TCO}_{\mathrm{month}} = C_{\mathrm{inf}} + C_{\mathrm{hw}} + C_{\mathrm{energy}} + C_{\mathrm{storage}} + C_{\mathrm{ops}}. \tag{8}$$

Формула (8) задаёт структуру совокупной стоимости владения; её параметризация выполняется на основе измерений, описанных в E5.

8.2 Условия экономической целесообразности

- **Выгоден:** $|\mathcal{C}_0| > \tau^\star$; $|\mathcal{A}| \geq 3$; доля ошибок, выявляемых дешёвой процедурой самопроверки, $\geq 30\%$; отсутствует доминирующий параллелизм.
- **Не выгоден:** $|\mathcal{C}_0| < \tau^\star$; истинная независимость подзадач; требование изоляции привилегий на процесс; модель плохо удерживает ролевой формат (измеряется в E2b).

Количественная оценка этих режимов проводится в экспериментах E1, E3 и E5.

9 Ограничения и угрозы валидности

9.1 Когда внутренняя интроспекция хуже внешней оркестрации

1. **Истинная независимость подзадач.** Параллельный запуск нескольких вызовов большой модели даёт линейное ускорение по времени ответа; внутренний цикл является последовательным.
2. **Необходимость изоляции привилегий.** Выполнение недоверенного кода и работа с секретами требуют процессных границ между ролями, что исключает внутренний режим.
3. **Слабое следование ролевому формату.** Если модель не удерживает ролевые префиксы, критик фактически превращается в генератор с иной инструкцией, и архитектурное преимущество исчезает. Этот риск оценивается в E2b.
4. **Систематическая ошибка критика.** Критик, опирающийся на собственные генерации, может устойчиво одобрять ошибочные кандидаты. Основным механизмом защиты в этом случае остаётся внешняя верификация.
5. **Компрессия, удаляющая критически важный факт.** Алгоритм 2 может исключить редкий, но существенный фрагмент; поэтому в сжатом контексте обязательно сохраняются трассы с ошибками и API-сигнатуры.

9.2 Угрозы валидности

Внутренняя валидность: на результаты могут влиять выбор seed и дрейф API; для уменьшения этого эффекта фиксируются идентификаторы версий моделей и используется набор из пяти значений seed. **Внешняя валидность:** HumanEval содержит короткие синтетические задачи, поэтому перенос выводов на реальные задачи программной инженерии дополнительно проверяется на

SWE-bench Lite в E4. **Конструктная валидность:** метрики U_{tok} и $Q_{\$}$ чувствительны к ценам токенов; этот фактор анализируется в E5 при вариации цен на $\pm 50\%$. **Статистическая валидность:** множественные сравнения учитываются поправкой Холма–Бонферрони, а при небольших n основной упор делается на доверительные интервалы.

9.3 Область применимости выводов

Полученные теоретические выводы относятся к задачам программной инженерии с длинным общим контекстом и не переносятся автоматически на иные классы задач. Промежуточные ролевые тексты рассматриваются как вычислительный инструмент, а не как интерпретируемое объяснение поведения модели [9]. Наконец, конкретные гиперпараметры $K = 3$, $I = 3$ и $\lambda = 0.5$ используются как рабочая конфигурация и могут быть дополнительно оптимизированы в последующих исследованиях.

10 Направления дальнейшего исследования

Дальнейшее развитие работы связано прежде всего с полной эмпирической валидацией предложенной архитектуры и уточнением условий её применимости.

10.1 Ближайшие задачи

R1. Реализовать воспроизводимый прототип конфигураций B0 и B3 со сквозным запуском на HumanEval.

R2. Провести пилотное сравнение на HumanEval в соответствии с протоколом E1 и получить первые оценки pass@1, токенного расхода и стоимости.

R3. Выполнить абляционный анализ E2a–E2d, включая оценку роли компрессии, критика, процедуры самопроверки и чувствительности метрик.

R4. Уточнить разграничение с Self-Refine либо путём усиления теоретического анализа, либо через прямое эмпирическое сравнение по U_{tok} на длинном контексте.

R5. Оценить TCO на реальных измерениях для выборки из $N = 500$ задач с доверительными интервалами и анализом чувствительности к ценам токенов.

R6. Калибровать веса таблицы 2 на основе экспертного опроса или дополнительных нормативных источников, сохранив анализ чувствительности.

10.2 Расширение экспериментальной программы

R7. Выполнить E3 и E4 в полном объёме и получить эмпирическую оценку порога $\tau^\star$ с доверительными интервалами.

R8. Расширить набор базовых конфигураций за счёт Reflexion [8], ReAct [6] и SWE-agent [17].

R9. Перенести оценку с SWE-bench Lite на SWE-bench Full на выборке не менее 300 задач.

R10. Провести экспертную оценку качества кода на подмножестве из 50 задач для валидации метрики M_i относительно профессионального суждения.

R11. Исследовать механизмы распространения ошибки и сравнить влияние ошибочного критика в Hybrid-INoT и Classical-MAS.

10.3 Целевые результаты эмпирической проверки

Центральный тезис о преимуществе внутренней ролевой сегрегации при длинном общем контексте может считаться подтверждённым при одновременном выполнении следующих условий:

1. E1: $\Delta U_{\text{tok}}^{B3-B2} \geq 15\%$ на $|\mathcal{C}_0| > 2048$, $p < 0.05$ после Holm–Bonferroni.
2. E1: pass@1 B3 не хуже B2 в пределах допуска на неухудшение 2 п.п.
3. E2a: каждый из компонентов (компрессия, критик, самопроверка) даёт значимый независимый вклад.
4. E3: $\tau^\star$ конечен и $< 16\,384$.
5. E4: $C_{\text{check}}^S < (\rho_0 - \rho_1) C_{\text{rerun}}^L$ на обоих датасетах с 95% CI.
6. E5: $\Delta\text{TCO} < 0$ при нижней границе CI, устойчиво к $\pm 50\%$ ценам.
7. R4: доказательство или эмпирическая значимость против Self-Refine.

При неполном выполнении этих условий объём выводов должен соотноситься с фактически полученными результатами.

10.4 Фокус исследования

Дальнейшие исследования следует сосредоточить на формальном и эмпирическом разграничении Hybrid-INoT и внешней мультиагентной оркестрации по токенной эффективности в задачах программной инженерии с длинным контекстом. Система метрик поддерживаемости и TCO-анализ при этом выступают как вспомогательные компоненты общей исследовательской программы.

11 Заключение

Статья формализует Hybrid-INoT как промежуточную архитектуру между внутриконтекстной самокритикой и внешней мультиагентной оркестрацией. Для этой архитектуры заданы состояние системы, функция маршрутизации и внутренний ролевой цикл; получено утверждение о токенном преимуществе при длинном контексте и числе ролей не менее трёх, выведена верхняя оценка расхода токенов в худшем случае и предложено условие экономической эффективности малой модели самопроверки. Кроме того, разработана система метрик для оценки токенной эффективности, стоимости и поддерживаемости кода и сформирована программа эмпирической валидации, необходимая для количественной проверки теоретических выводов. В совокупности эти результаты задают формальную основу для дальнейшего сопоставления Hybrid-INoT с Classical-MAS и родственными подходами на задачах программной инженерии.

References

[1] H. Sun and S. Zeng, “Introspection of Thought Helps AI Agents,” arXiv preprint arXiv:2507.08664, 2025.

[2] A. J. Ko, R. DeLine, and G. Venolia, "Information Needs in Collocated Software Development Teams," in *Proc. 29th Int. Conf. Software Engineering*, 2007, pp. 344–353.

[3] C. Parnin and S. Rugaber, "Resumption Strategies for Interrupted Programming Tasks," *Software Quality Journal*, vol. 19, no. 1, pp. 5–34, 2011.

[4] M. K. Goncalves, C. R. B. de Souza, and V. M. Gonzalez, "Collaboration, Information Seeking and Communication," *J. Universal Computer Science*, vol. 17, no. 11, pp. 1529–1550, 2011.

[5] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *NeurIPS*, vol. 35, 2022, pp. 24824–24837.

[6] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," *ICLR*, 2023.

[7] A. Madaan *et al.*, "Self-Refine: Iterative Refinement with Self-Feedback," *NeurIPS*, 2023.

[8] N. Shinn *et al.*, "Reflexion: Language Agents with Verbal Reinforcement Learning," *NeurIPS*, 2023.

[9] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting," *NeurIPS*, 2023.

[10] G. Weiss, *Multiagent Systems*. MIT Press, 1999.

[11] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley, 2009.

[12] S. Hong *et al.*, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," *ICLR*, 2024.

[13] C. Qian *et al.*, "ChatDev: Communicative Agents for Software Development," *ACL*, 2024.

[14] T. Guo *et al.*, "Large Language Model based Multi-Agents: A Survey of Progress and Challenges," arXiv:2402.01680, 2024.

[15] C. Qian *et al.*, "Scaling Large Language Model-based Multi-Agent Collaboration," arXiv:2406.07155, 2024.

[16] C. E. Jimenez *et al.*, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," *ICLR*, 2024.

[17] J. Yang *et al.*, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," *NeurIPS*, 2024.

[18] M. Chen *et al.*, "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.

[19] J. Austin *et al.*, "Program Synthesis with Large Language Models," arXiv:2108.07732, 2021.

[20] H. Jiang *et al.*, "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression," *ACL*, 2024.

[21] Z. Pan *et al.*, "LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression," *ACL Findings*, 2024.

[22] H. Jia, E. T. Barr, and S. Mechtaev, "Compressing Code Context for LLM-based Issue Resolution," arXiv:2603.28119, 2026.

[23] Z. Bi *et al.*, "Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback," arXiv:2403.16792, 2024.

[24] C.-Y. Su and C. McMillan, "Do Code LLMs Do Static Analysis?," arXiv:2505.12118, 2025.

[25] M. Sepidband, H. Taherkhani, S. Wang, and H. Hemmati, "Enhancing LLM-Based Code Generation with Complexity Metrics," arXiv:2505.23953, 2025.

[26] ISO/IEC 25010:2023, "Systems and software engineering — SQuaRE — Product quality model," ISO, 2023.

[27] T. J. McCabe, "A Complexity Measure," *IEEE Trans. Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.